Server Routes Differences with PWA Kit

PWA Kit server routes are typically centralized as Express handlers in SSR server code. Storefront Next replaces that with file-based resource routes and loader/action handlers using standard Web `Request` / `Response` APIs. This distributes server behavior across route modules and aligns API endpoints with routing conventions.

Key Differences

Aspect	PWA Kit	Storefront Next
Pattern	Express routes in `ssr.js`	File-based resource ro
Configuration	Centralized in one file	Distributed across rou
Request API	Express `req` / `res` objects	Web Standard `Request` / `Response`
Route Definition	`app.get('/path', handler)`	Export `loader` / `acti` file
Dynamic Routes	Express params (`/:param`)	File naming (`$param`
Middleware	Express middleware chain	React Router middlew
Client Execution	Server only	Optional `clientLoader` / `cli`

PWA Kit Express Routes

```
// Single file: app/ssr.js
app.get("/api/search", async (req, res) => {
  const query = req.query.q;
  const results = await searchProducts(query);
  res.json(results);
});

```

Storefront Next Resource Routes

```
// src/routes/resource.search.ts
export function loader({ request }: LoaderFunctio
  const url = new URL(request.url);
  const query = url.searchParams.get("q");
  const results = await searchProducts(query);
  return Response.json(results);
}

// src/routes/resource.auth.login.ts
export async function action({ request }: ActionF
  const { email, password } = await request.json(
  const session = await login(email, password);
  return Response.json(session);
}
```

Key Similarities

- Both support GET (loaders) and POST/PUT/DELETE (actions).
- Both can set cache headers and response status codes.
- Both integrate with SLAS for authentication callbacks.
- Both can proxy Commerce API calls.
- Both support dynamic route parameters.

Storefront Conversion Considerations

Keep these tips in mind when you convert your PWA Kit storefront to Storefront Next.

- Route extraction: Move logic from `ssr.js` callbacks into separate route files.
- Request handling: Convert Express `req.body` / `req.query` to `request.json()` / `URL.searchParams`.
- Response handling: Convert `res.json()` to `Response.json()`.
- Middleware: Use React Router middleware exports instead of Express middleware.
- Client-side option: Consider adding `clientLoader` for endpoints that can run on the client.